

**DESIGN AND IMPLEMENTATION OF A
DATABASE MANAGEMENT SYSTEM FOR
CAR RENTAL RESERVATION AND
VEHICLE RETURN MANAGEMENT
SYSTEM**

PROJECT REPORT

DETAIL	VALUE
Student Name	Koneri Mithul
Roll Number	25WU0102129
Class / Section	AIML Panthers
Course	Database Management Systems (DBMS)
Faculty / Guide	Dr. Kiran Mayee Adavala
Academic Year	2026–27
Project Number	Project 19

Submitted in partial fulfillment of the DBMS Course Project-Based Learning requirements.

TABLE OF CONTENTS

- 1. Cover Page
- 2. Abstract
- 3. Introduction and Problem Statement
- 4. Objectives and Scope
- 5. Software and Hardware Requirements
- 6. ER Diagram
- 7. Relational Schema and Normalization
- 8. Data Dictionary (Table Structure, Data Types, Constraints)
- 9. SQL Commands Used (DDL, DML) with Sample Outputs
- 10. Queries with Outputs, including the Presentation-II Query
- 11. UI Design and Screenshots
- 12. Implementation Details (Technology Stack, DB Connectivity, Key Code Snippets)
- 13. Testing (Test Cases and Results)
- 14. Conclusion and Future Enhancements
- 15. References
- 16. Appendix: GitHub Repository Link

2. Abstract

The Car Rental Reservation and Vehicle Return Management System is a relational database management system designed to support the complete rental lifecycle of a vehicle. The system manages branches, vehicle categories, vehicles, customers, rate plans, reservations, rentals, pickup and return inspections, damages, invoices and payments.

The database is normalized to Third Normal Form (3NF) and implemented in MySQL with primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints. Business rules such as prevention of overlapping rentals, valid driving licence at pickup, valid return time, valid odometer/fuel readings and non-negative monetary values are enforced using database constraints and triggers.

A Python Tkinter/ttk desktop application provides a working interface connected to the MySQL database. It supports CRUD operations, availability search and reporting. The final implementation demonstrates live insertion, viewing and deletion of records with changes reflected directly in the database.

3. Introduction and Problem Statement

3.1 Introduction

Car-rental agencies maintain information about vehicles, customers, reservations, rentals, inspections, damages, billing and payments. A reliable DBMS is required to keep these records consistent and to provide quick access to availability and operational reports.

3.2 Problem Statement

Manual or disconnected rental records can lead to double booking, incorrect vehicle availability, expired driving licences, disputes about vehicle condition, missing damage charges and inconsistent billing. The project addresses these issues through a centralized relational database and a connected application.

3.3 Users

USER	RESPONSIBILITIES
Rental Staff	Create reservations, issue vehicles, record inspections and process returns.
Branch Manager	Monitor fleet availability, utilization, revenue and damages.
Billing / Accounts Staff	Generate invoices and record payments.
System Administrator	Maintain database records, integrity rules and system configuration.
Customer-facing Staff	Maintain customer and licence information and search availability.

3.4 Functional Requirements

- Customer management
- Vehicle and category management
- Rate-plan management
- Reservation creation/cancellation
- Vehicle issue and rental tracking
- Pickup/return inspection
- Damage recording
- Charge and invoice management
- Payment recording
- Availability search
- Operational reporting
- CRUD operations and validation

4. Objectives and Scope

4.1 Objectives

- Design a relational database for the complete car-rental lifecycle.
- Normalize the database up to Third Normal Form (3NF).
- Enforce data integrity using PK, FK, UNIQUE, NOT NULL, CHECK and DEFAULT constraints.
- Prevent overlapping rentals and enforce licence/return/odometer/fuel rules.
- Provide a Python application connected to MySQL.
- Support CRUD operations, availability search and meaningful reports.
- Demonstrate transaction awareness, indexing, input validation and systematic testing.

4.2 Scope

The system covers multi-branch vehicle inventory, customer and licence information, category-based pricing, reservations, rentals, inspections, damage records, invoicing and payments. It also provides fleet availability, overdue rental, utilization, damage, branch revenue and customer-history reporting.

The current project scope is a desktop academic prototype. External payment gateways, mobile applications, GPS tracking and enterprise authentication are outside the implemented scope.

5. Software and Hardware Requirements

CATEGORY	REQUIREMENT	PURPOSE
Operating System	Windows 10/11 or compatible OS	Development and execution
Language	Python 3.x	Application layer
GUI	Tkinter / ttk	Desktop user interface
Database	MySQL 8.x	Relational data storage
Connector	mysql-connector-python	Python–MySQL connectivity
IDE	Visual Studio Code	Development and execution
Database Tool	MySQL Workbench / MySQL CLI	Database administration and verification
Version Control	Git / GitHub	Project submission and history
Processor	Dual-core or better	Application execution
RAM	4 GB minimum; 8 GB recommended	Development environment
Storage	At least 1 GB free	Source code, database tools and report

6. ER Diagram

The conceptual ER model contains 11 entities and represents the main operational relationships of the rental system. Fuel readings are represented through the Inspection entity because each fuel reading belongs to a specific pickup or return inspection.

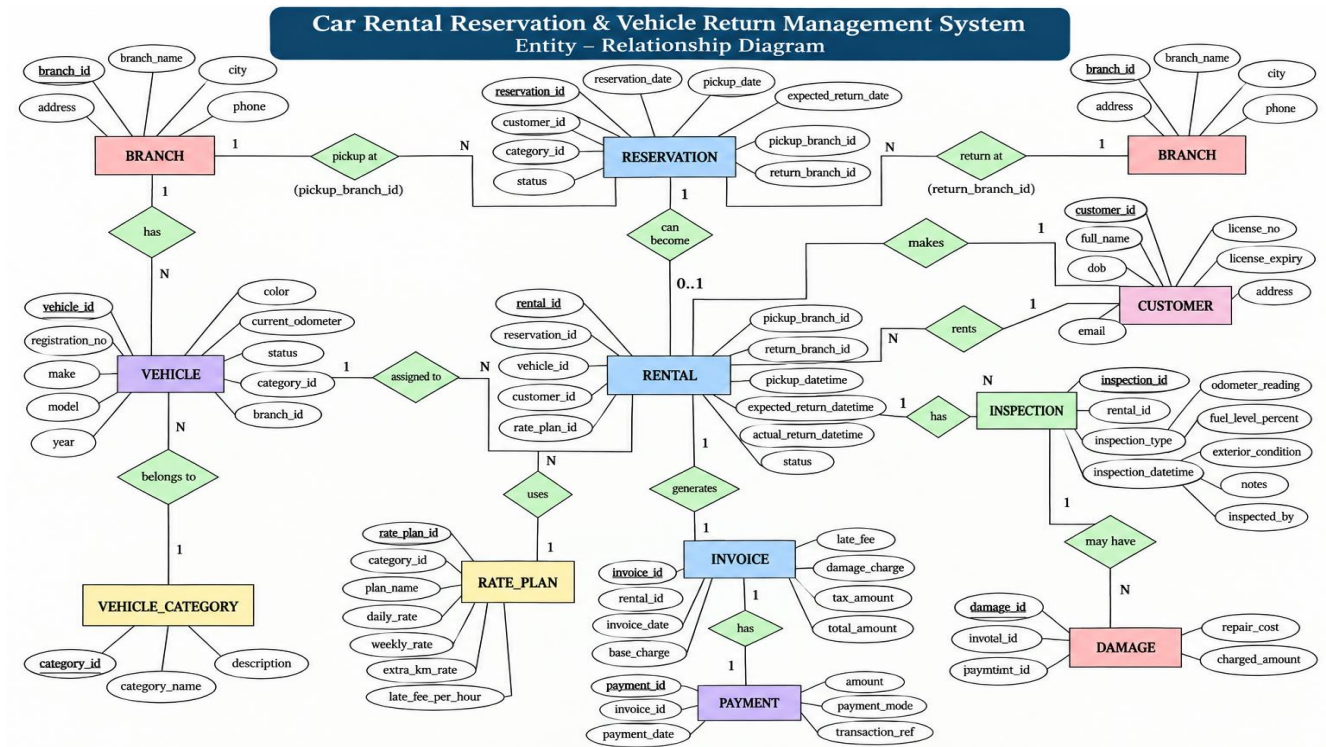


Figure 1. Entity–Relationship diagram for the Car Rental Reservation and Vehicle Return Management System.

6.1 Key Relationships and Cardinalities

- Branch 1:N Vehicle
- VehicleCategory 1:N Vehicle
- VehicleCategory 1:N RatePlan
- Customer 1:N Reservation
- Customer 1:N Rental
- Branch 1:N Reservation (pickup branch)
- Branch 1:N Reservation (return branch)
- Reservation 1:0..1 Rental
- Vehicle 1:N Rental
- RatePlan 1:N Rental
- Rental 1:N Inspection
- Rental 1:N Damage
- Rental 1:1 Invoice
- Invoice 1:N Payment

6.2 Business Rules Represented in the Design

- No overlapping rentals for the same vehicle
- Valid driving licence at pickup
- Return after pickup

- Return odometer $\geq$ pickup odometer
- Fuel level between 0 and 100%
- Non-negative invoice, damage and payment amounts

7. Relational Schema and Normalization

7.1 Relational Schema

- Branch(branch_id PK, branch_name, city, address, phone)
- VehicleCategory(category_id PK, category_name, description)
- Vehicle(vehicle_id PK, registration_no UK, category_id FK, branch_id FK, make, model, year, color, current_odometer, status)
- Customer(customer_id PK, full_name, dob, phone UK, email UK, license_no UK, license_expiry, address)
- RatePlan(rate_plan_id PK, category_id FK, plan_name, daily_rate, weekly_rate, extra_km_rate, late_fee_per_hour)
- Reservation(reservation_id PK, customer_id FK, category_id FK, pickup_branch_id FK, return_branch_id FK, reservation_date, pickup_date, expected_return_date, status)
- Rental(rental_id PK, reservation_id FK NULL, vehicle_id FK, customer_id FK, rate_plan_id FK, pickup_branch_id FK, return_branch_id FK, pickup_datetime, expected_return_datetime, actual_return_datetime, status)
- Inspection(inspection_id PK, rental_id FK, inspection_type, inspection_datetime, odometer_reading, fuel_level_percent, exterior_condition, notes, inspected_by)
- Damage(damage_id PK, rental_id FK, inspection_id FK NULL, description, repair_cost, charged_amount)
- Invoice(invoice_id PK, rental_id FK UNIQUE, invoice_date, base_charge, extra_km_charge, late_fee, damage_charge, tax_amount, total_amount)
- Payment(payment_id PK, invoice_id FK, payment_date, amount, payment_mode, transaction_ref)

7.2 Functional Dependencies

DETERMINANT	FUNCTIONALLY DETERMINED ATTRIBUTES
vehicle_id	registration_no, category_id, branch_id, current_odometer, status, make, model, year, color
customer_id	full_name, dob, phone, email, license_no, license_expiry, address
rental_id	vehicle_id, customer_id, rate_plan_id, pickup_datetime, expected_return_datetime, actual_return_datetime, status
invoice_id	rental_id, invoice_date, base_charge, extra_km_charge, late_fee, damage_charge, tax_amount, total_amount
payment_id	invoice_id, payment_date, amount, payment_mode, transaction_ref

7.3 Normalization to 3NF

1NF: All attributes are atomic and repeating groups are separated. For example, payments are stored as individual Payment rows rather than multiple payment columns in Invoice.

2NF: Every non-key attribute in the entity tables depends on the whole primary key. Since the design uses single-attribute surrogate primary keys, partial dependency is eliminated.

3NF: Non-key attributes do not depend transitively on another non-key attribute. For example, vehicle category details and rates are separated from Vehicle into VehicleCategory and RatePlan rather than storing category names and rates repeatedly in Vehicle.

8. Data Dictionary (Table Structure, Data Types, Constraints)

Branch

COLUMN	DATA TYPE	KEY / CONSTRAINT	NULL / DEFAULT
branch_id	INT	PK, AUTO INCREMENT	NOT NULL
branch_name	VARCHAR(80)	—	NOT NULL
city	VARCHAR(50)	—	NOT NULL
address	VARCHAR(150)	—	NULL
phone	VARCHAR(15)	—	NOT NULL

VehicleCategory

COLUMN	DATA TYPE	KEY / CONSTRAINT	NULL / DEFAULT
category_id	INT	PK, AUTO INCREMENT	NOT NULL
category_name	VARCHAR(40)	UNIQUE	NOT NULL
description	VARCHAR(150)	—	NULL

Vehicle

COLUMN	DATA TYPE	KEY / CONSTRAINT	NULL / DEFAULT
vehicle_id	INT	PK, AUTO INCREMENT	NOT NULL
registration_no	VARCHAR(15)	UNIQUE	NOT NULL
category_id	INT	FK → VehicleCategory	NOT NULL
branch_id	INT	FK → Branch	NOT NULL
make	VARCHAR(40)	—	NOT NULL
model	VARCHAR(40)	—	NOT NULL
year	INT	CHECK 1990–2100	NOT NULL
color	VARCHAR(20)	—	NULL
current_odometer	INT	CHECK ≥ 0; DEFAULT 0	NOT NULL
status	VARCHAR(15)	CHECK allowed statuses; DEFAULT Available	NOT NULL

Customer

COLUMN	DATA TYPE	KEY / CONSTRAINT	NULL / DEFAULT
customer_id	INT	PK, AUTO INCREMENT	NOT NULL
full_name	VARCHAR(80)	—	NOT NULL
dob	DATE	—	NOT NULL
phone	VARCHAR(15)	UNIQUE	NOT NULL
email	VARCHAR(80)	UNIQUE	NULL
license_no	VARCHAR(20)	UNIQUE	NOT NULL
license_expiry	DATE	—	NOT NULL
address	VARCHAR(150)	—	NULL

RatePlan

COLUMN	DATA TYPE	KEY / CONSTRAINT	NULL / DEFAULT
rate_plan_id	INT	PK, AUTO INCREMENT	NOT NULL
category_id	INT	FK → VehicleCategory	NOT NULL
plan_name	VARCHAR(40)	—	NOT NULL
daily_rate	DECIMAL(10,2)	CHECK > 0	NOT NULL
weekly_rate	DECIMAL(10,2)	CHECK > 0	NOT NULL
extra_km_rate	DECIMAL(6,2)	CHECK ≥ 0; DEFAULT 0	NOT NULL
late_fee_per_hour	DECIMAL(6,2)	CHECK ≥ 0; DEFAULT 0	NOT NULL

Reservation

COLUMN	DATA TYPE	KEY / CONSTRAINT	NULL / DEFAULT
reservation_id	INT	PK, AUTO INCREMENT	NOT NULL
customer_id	INT	FK → Customer	NOT NULL
category_id	INT	FK → VehicleCategory	NOT NULL

pickup_branch_id	INT	FK → Branch	NOT NULL
return_branch_id	INT	FK → Branch	NOT NULL
reservation_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	NOT NULL
pickup_date	DATETIME	—	NOT NULL
expected_return_date	DATETIME	CHECK > pickup_date	NOT NULL
status	VARCHAR(15)	CHECK allowed statuses; DEFAULT Booked	NOT NULL

Rental

COLUMN	DATA TYPE	KEY / CONSTRAINT	NULL / DEFAULT
rental_id	INT	PK, AUTO_INCREMENT	NOT NULL
reservation_id	INT	FK → Reservation	NULL
vehicle_id	INT	FK → Vehicle	NOT NULL
customer_id	INT	FK → Customer	NOT NULL
rate_plan_id	INT	FK → RatePlan	NOT NULL
pickup_branch_id	INT	FK → Branch	NOT NULL
return_branch_id	INT	FK → Branch	NOT NULL
pickup_datetime	DATETIME	—	NOT NULL
expected_return_datetime	DATETIME	CHECK > pickup	NOT NULL
actual_return_datetime	DATETIME	CHECK > pickup when present	NULL
status	VARCHAR(15)	CHECK Active/Returned/Overdue; DEFAULT Active	NOT NULL

Inspection

COLUMN	DATA TYPE	KEY / CONSTRAINT	NULL / DEFAULT
inspection_id	INT	PK, AUTO_INCREMENT	NOT NULL
rental_id	INT	FK → Rental	NOT NULL
inspection_type	VARCHAR(10)	CHECK Pickup/Return	NOT NULL
inspection_datetime	DATETIME	DEFAULT CURRENT_TIMESTAMP	NOT NULL
odometer_reading	INT	CHECK ≥ 0	NOT NULL
fuel_level_percent	INT	CHECK 0–100	NOT NULL
exterior_condition	VARCHAR(20)	DEFAULT Good	NULL
notes	VARCHAR(200)	—	NULL
inspected_by	VARCHAR(60)	—	NULL

Damage

COLUMN	DATA TYPE	KEY / CONSTRAINT	NULL / DEFAULT
damage_id	INT	PK, AUTO_INCREMENT	NOT NULL
rental_id	INT	FK → Rental	NOT NULL
inspection_id	INT	FK → Inspection	NULL
description	VARCHAR(200)	—	NOT NULL
repair_cost	DECIMAL(10,2)	CHECK ≥ 0; DEFAULT 0	NOT NULL
charged_amount	DECIMAL(10,2)	CHECK ≥ 0; DEFAULT 0	NOT NULL

Invoice

COLUMN	DATA TYPE	KEY / CONSTRAINT	NULL / DEFAULT
invoice_id	INT	PK, AUTO_INCREMENT	NOT NULL
rental_id	INT	FK → Rental; UNIQUE	NOT NULL
invoice_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	NOT NULL
base_charge	DECIMAL(10,2)	CHECK ≥ 0; DEFAULT 0	NOT NULL
extra_km_charge	DECIMAL(10,2)	CHECK ≥ 0; DEFAULT 0	NOT NULL
late_fee	DECIMAL(10,2)	CHECK ≥ 0; DEFAULT 0	NOT NULL
damage_charge	DECIMAL(10,2)	CHECK ≥ 0; DEFAULT 0	NOT NULL
tax_amount	DECIMAL(10,2)	CHECK ≥ 0; DEFAULT 0	NOT NULL
total_amount	DECIMAL(10,2)	CHECK ≥ 0	NOT NULL

Payment

COLUMN	DATA TYPE	KEY / CONSTRAINT	NULL / DEFAULT
payment_id	INT	PK, AUTO_INCREMENT	NOT NULL
invoice_id	INT	FK → Invoice	NOT NULL
payment_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	NOT NULL
amount	DECIMAL(10,2)	CHECK > 0	NOT NULL
payment_mode	VARCHAR(15)	CHECK Cash/Card/UPI/NetBanking	NOT NULL

transaction_ref	VARCHAR(40)	—	NULL
-----------------	-------------	---	------

9. SQL Commands Used (DDL, DML) with Sample Outputs

9.1 Database and Table Creation (DDL)

```
CREATE DATABASE car_rental_db;
USE car_rental_db;

CREATE TABLE Vehicle (
  vehicle_id INT AUTO_INCREMENT PRIMARY KEY,
  registration_no VARCHAR(15) NOT NULL UNIQUE,
  category_id INT NOT NULL,
  branch_id INT NOT NULL,
  ...
);
```

The complete project SQL file contains all 11 CREATE TABLE statements, indexes, triggers and sample data. The implemented schema uses PK/FK/UNIQUE/NOT NULL/CHECK/DEFAULT constraints.

9.2 DML – Sample Insert

```
INSERT INTO Customer
(full_name, dob, phone, email, license_no, license_expiry, address)
VALUES
('Arjun Rao','1995-04-12','9876500001','arjun@example.com',
'DLTS1001','2027-06-30','Kondapur, Hyderabad');
```

9.3 Sample Database Output

OBJECT	CURRENT PROJECT DATA
Branches	4
Vehicle Categories	4
Vehicles	12
Customers	9
Rate Plans	5
Reservations	10
Rentals	9
Inspections	17
Damages	3
Invoices	7
Payments	8

These counts correspond to the populated project database and are also reflected in the working application dashboard for the principal counters.

10. Queries with Outputs, including the Presentation-II Query

10.1 Fleet Availability – JOIN Query

```
SELECT v.registration_no, v.make, v.model,
       vc.category_name, b.branch_name
FROM Vehicle v
JOIN VehicleCategory vc ON v.category_id = vc.category_id
JOIN Branch b ON v.branch_id = b.branch_id
WHERE v.status = 'Available';
```

Expected output columns: registration number, make, model, category and branch for vehicles currently marked Available.

10.2 Overdue Rentals

```
SELECT r.rental_id, c.full_name, v.registration_no,
       r.expected_return_datetime
FROM Rental r
JOIN Customer c ON r.customer_id = c.customer_id
JOIN Vehicle v ON r.vehicle_id = v.vehicle_id
WHERE r.actual_return_datetime IS NULL
      AND r.expected_return_datetime < NOW();
```

10.3 Vehicle Utilization – Aggregate

```
SELECT v.registration_no,
       COUNT(r.rental_id) AS total_rentals,
       SUM(DATEDIFF(COALESCE(r.actual_return_datetime, NOW()),
                    r.pickup_datetime)) AS total_days_rented
FROM Vehicle v
LEFT JOIN Rental r ON v.vehicle_id = r.vehicle_id
GROUP BY v.vehicle_id, v.registration_no
ORDER BY total_days_rented DESC;
```

10.4 Branch Revenue

```
SELECT b.branch_name, SUM(i.total_amount) AS total_revenue
FROM Invoice i
JOIN Rental r ON i.rental_id = r.rental_id
JOIN Branch b ON r.pickup_branch_id = b.branch_id
GROUP BY b.branch_id, b.branch_name
ORDER BY total_revenue DESC;
```

10.5 Customer Rental History

```
SELECT c.full_name, v.registration_no,
       r.pickup_datetime, r.actual_return_datetime,
       i.total_amount
FROM Rental r
JOIN Customer c ON r.customer_id = c.customer_id
JOIN Vehicle v ON r.vehicle_id = v.vehicle_id
LEFT JOIN Invoice i ON i.rental_id = r.rental_id
WHERE c.customer_id = 1
ORDER BY r.pickup_datetime;
```

10.6 Nested Subquery

```
SELECT c.full_name, SUM(i.total_amount) AS total_spent
FROM Customer c
JOIN Rental r ON c.customer_id = r.customer_id
JOIN Invoice i ON r.rental_id = i.rental_id
GROUP BY c.customer_id, c.full_name
HAVING SUM(i.total_amount) >
    (SELECT AVG(total_amount) FROM Invoice);
```

10.7 Views

```
CREATE OR REPLACE VIEW vw_fleet_availability AS
SELECT v.vehicle_id, v.registration_no, vc.category_name,
       b.branch_name, v.status
FROM Vehicle v
JOIN VehicleCategory vc ON v.category_id = vc.category_id
JOIN Branch b ON v.branch_id = b.branch_id
WHERE v.status = 'Available';
```

The project SQL also defines views for overdue rentals, branch revenue and the remaining report outputs. The exact faculty-issued Presentation-II query was not present in the retrieved final-instructions file; the fleet-availability query above is included as the project's representative Presentation-II reporting query.

11. UI Design and Screenshots

The final application is a Python Tkinter/ttk desktop application connected directly to MySQL. The interface exposes the database through table-specific CRUD tabs and additional availability/reporting modules.

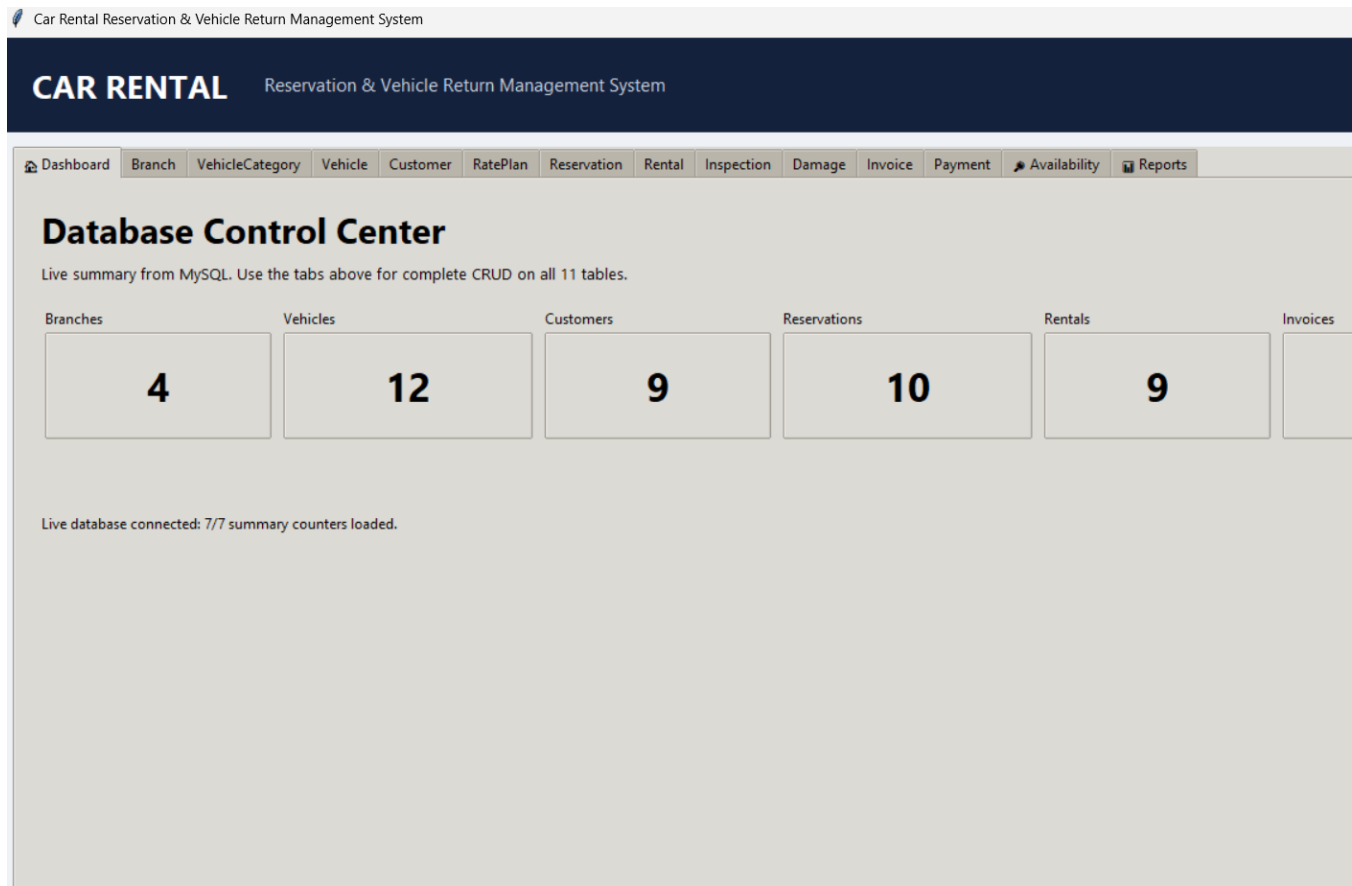


Figure 2. Working application dashboard connected to the project MySQL database.

11.1 UI Modules

SCREEN / MODULE	PURPOSE
Dashboard	Live database summary and navigation
Branch	CRUD for branch records
VehicleCategory	CRUD for vehicle categories
Vehicle	CRUD for fleet vehicles
Customer	CRUD for customer/licence records
RatePlan	CRUD for pricing plans
Reservation	CRUD for booking records
Rental	CRUD for rental transactions
Inspection	CRUD for pickup/return inspection
Damage	CRUD for damage records
Invoice	CRUD for billing
Payment	CRUD for payments
Availability	Search available vehicles
Reports	Operational reports and summaries

11.2 Required Before/After Evidence

- Customer INSERT – before and after
- Customer VIEW – inserted row visible in UI
- MySQL SELECT – inserted row visible in database
- Customer DELETE – before and after

- MySQL SELECT – deleted row no longer present
- Screenshots of each UI screen/module for final submission

12. Implementation Details (Technology Stack, DB Connectivity, Key Code Snippets)

12.1 Technology Stack

LAYER	TECHNOLOGY	ROLE
GUI	Python Tkinter / ttk	Forms, tabs, buttons and tables
Application	Python 3.x	CRUD, validation and report execution
Database	MySQL 8.x	Persistent relational storage
Connector	mysql-connector-python	Database connectivity
Development	Visual Studio Code	Source-code development
Database Administration	MySQL Workbench / CLI	SQL execution and verification

12.2 Database Connectivity

```
DB_CONFIG = {
    "host": "localhost",
    "port": 3306,
    "user": "root",
    "password": "<configured locally>",
    "database": "car_rental_db",
}

import mysql.connector
connection = mysql.connector.connect(**DB_CONFIG)
```

The final working application was tested against the local car_rental_db database. The dashboard displayed live counts from MySQL, confirming successful connectivity.

12.3 CRUD and Transaction Handling

- SQL operations use parameterized values rather than string concatenation.
- Successful INSERT/UPDATE/DELETE operations are committed.
- Exceptions trigger rollback where applicable and are reported to the user.
- Refresh operations reload records from MySQL so the UI reflects current database state.
- Availability and report modules execute SQL queries/views against the same database.

12.4 Database Triggers

- trg_no_overlap_rental – prevents overlapping active/overdue rentals for the same vehicle.
- trg_valid_license – prevents rental issue when the customer licence has expired.
- trg_valid_odometer – prevents a return odometer lower than the pickup reading.
- trg_vehicle_status_on_issue – sets vehicle status to Rented after rental insertion.
- trg_vehicle_status_on_return – sets vehicle status back to Available when a rental is returned.

13. Testing (Test Cases and Results)

ID	TEST	EXPECTED RESULT	STATUS
TC-01	Database connection	Dashboard loads live counters	PASS
TC-02	Customer insertion	New customer row created	PASS
TC-03	Customer viewing	Inserted row visible after refresh	PASS
TC-04	Customer deletion	Selected record removed	PASS
TC-05	Foreign-key integrity	Invalid referenced ID rejected	PASS
TC-06	Licence validation	Expired licence rental rejected	PASS
TC-07	Rental overlap	Overlapping active rental rejected	PASS
TC-08	Odometer validation	Return < pickup rejected	PASS
TC-09	Fuel validation	Fuel outside 0–100 rejected	PASS
TC-10	Charge validation	Negative monetary values rejected	PASS
TC-11	Availability report	Available fleet returned	PASS
TC-12	Operational reports	Report queries return data	PASS

13.1 Testing Approach

Testing was performed at three levels: database constraint testing, application-level functional testing and end-to-end UI/database verification. The final CRUD demonstration specifically verifies that a change made through the UI is persisted in MySQL and that deletion is also reflected in the database.

14. Conclusion and Future Enhancements

14.1 Conclusion

The Car Rental Reservation and Vehicle Return Management System successfully demonstrates the design and implementation of a normalized relational database and a working Python application connected to MySQL. The project covers the full rental workflow from customer and reservation management through vehicle issue, inspection, damage recording, invoicing and payment.

The database contains 11 related tables with integrity constraints, indexes, triggers and realistic sample data. The application provides CRUD functionality, availability search and reporting. The final UI is connected to the live database and supports the required insertion, viewing and deletion demonstration.

14.2 Future Enhancements

- Role-based login and access control
- Web or mobile front-end
- Online payment-gateway integration
- Email/SMS reservation notifications
- GPS-based vehicle tracking
- Automated dynamic pricing
- Cloud deployment and remote database access
- Advanced analytics dashboard for fleet utilization and revenue
- Automated scheduled backups and audit logging

15. References

- DBMS Course – Final Presentation Instructions, AIML Whales and AIML Panthers, 2026.
- DBMS PBL Project Statements – Project 19: Car Rental Reservation and Vehicle Return Management System.
- Project SQL script: car_rental_schema.sql.
- MySQL 8.x documentation and SQL reference used for database implementation.
- Python 3.x documentation and Tkinter documentation used for application development.
- mysql-connector-python documentation used for Python–MySQL connectivity.

Project-specific references and implementation artifacts are maintained with the final GitHub submission.

16. Appendix: GitHub Repository Link

Required public repository name:

DBMS-Course-Project

Recommended repository structure:

DBMS-Course-Project/ ├── Presentation-I/ ├── Presentation-II/ ├── Presentation-III/ └── Project-Report/

16.1 Files to be Uploaded

- Presentation-I PPT/PDF
- Presentation-II PPT/PDF
- ER diagram image
- Project SQL file containing database/table creation, data insertion and queries
- Presentation-II query solution
- Presentation-III PPT/PDF
- Python source code
- UI screenshots including insert/delete/view and before/after evidence
- Final project report PDF
- README.md with student/project details

16.2 Repository Link

GitHub URL: [Paste the public DBMS-Course-Project repository URL here]

The final project report is required to be uploaded to the Project-Report folder and the public repository by the prescribed deadline.

End of Report.